

The Complete Guide to Data Structures and Algorithms in Python

Table of Contents

- [Introduction and Mindset](#)
- [Problem-Solving Framework](#)
- [Time and Space Complexity](#)
- [Core Data Structures](#)
- [Essential Algorithms](#)
- [LeetCode 75: The Optimal Path](#)
- [Problem Patterns to Master](#)
- [Weekly Practice Schedule](#)
- [Resources and Next Steps](#)

Introduction and Mindset

The Engineer's Mindset

Becoming proficient in Data Structures and Algorithms isn't just about memorizing solutions—it's about developing a systematic thought process. As experienced engineers know, the key is breaking down complex problems into smaller, manageable pieces.

The 5 Principles of DSA Problem Solving:

1. **Understand before coding:** Never jump straight to coding. Ensure you fully understand the problem.
2. **Start with brute force:** Always get a working solution first, then optimize.
3. **Pattern recognition:** Similar problems have similar solutions.
4. **Test edge cases:** Empty arrays, single elements, negative values, etc.
5. **Iterate and improve:** Your first solution is rarely your best.

Why LeetCode 75 vs. LeetCode 150?

While the LeetCode 150 covers more ground, the LeetCode 75 is carefully curated to provide maximum learning with minimal redundancy. It covers all the essential patterns without overwhelming you with similar problems. Master these 75, and you'll have the tools to solve hundreds more.

Problem-Solving Framework

For every problem, follow this systematic approach:

1. Understand the Problem

- What are the inputs and their constraints?
- What is the expected output?
- Can I restate the problem in my own words?
- Are there any edge cases to consider?

2. Explore Examples

- Start with simple examples
- Progress to more complex cases
- Consider edge cases (empty inputs, large values, etc.)

3. Break Down the Approach

- Think out loud
- Consider multiple approaches
- Identify the data structures needed
- Sketch the algorithm steps

4. Code the Solution

- Write clean, readable code
- Use meaningful variable names
- Add comments for complex logic

5. Test and Debug

- Trace through with example inputs
- Check edge cases
- Identify and fix bugs

6. Analyze and Optimize

- What's the time complexity?
- What's the space complexity?
- Can it be improved?

Time and Space Complexity

Understanding complexity is crucial for writing efficient code:

Common Time Complexities

- **O(1)**: Constant time
- **O(log n)**: Logarithmic time (binary search)
- **O(n)**: Linear time (single loop)
- **O(n log n)**: Linearithmic time (efficient sorting)
- **O(n²)**: Quadratic time (nested loops)
- **O(2ⁿ)**: Exponential time (naive recursive solutions)

Python-Specific Operations Complexity

Operation	Complexity	Notes
List index/append/pop	O(1)	At the end
List insert/remove	O(n)	Requires shifting elements
Dictionary get/set/delete	O(1)	Average case
Set add/remove/contains	O(1)	Average case
Sorting (list.sort())	O(n log n)	TimSort algorithm
len()	O(1)	For all built-in collections
min()/max()	O(n)	Scans entire collection
String concatenation	O(n)	Creates new string
List comprehension	O(n)	Single loop

Core Data Structures

Arrays/Lists

Python lists are versatile and foundational:

python

Creating and accessing

```
nums = [1, 2, 3, 4, 5]
```

```
first = nums[0] # O(1) access
```

```
last = nums[-1] # O(1) access
```

Common operations

```
nums.append(6) # O(1) amortized
```

```
nums.insert(0, 0) # O(n) - requires shifting elements
```

```
nums.pop() # O(1) from end
```

```
nums.pop(0) # O(n) from beginning - requires shifting
```

```
nums.remove(3) # O(n) - need to find element first
```

Slicing

```
first_three = nums[:3] # O(k) where k is slice size
```

```
reversed_list = nums[::-1] # O(n)
```

List comprehension

```
squares = [x*x for x in nums] # O(n)
```

```
evens = [x for x in nums if x % 2 == 0] # O(n)
```

When to use:

- When order matters
- When you need random access
- When you need a simple, iterable collection

Common pitfalls:

- Costly insertions/deletions at the beginning
- Inefficient searches without sorting
- High memory overhead

Example Problem: Two Sum

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

python

```
def two_sum(nums, target):
    # Brute force: O(n^2) time, O(1) space
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]

    # Optimized: O(n) time, O(n) space
    seen = {} # value -> index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i

    return [] # No solution found
```

Key insight: Trading space for time using a hash map to store values we've seen.

Strings

Strings in Python are immutable sequences of characters:

python

```
# Creating and accessing
s = "hello world"
first_char = s[0] # O(1)

# Common operations
length = len(s) # O(1)
uppercase = s.upper() # O(n)
words = s.split() # O(n)

# String methods
s.startswith("hello") # O(1) to O(n)
s.find("world") # O(n)
s.replace("world", "python") # O(n)

# String joining
words = ["hello", "world"]
sentence = " ".join(words) # O(n)
```

When to use:

- Text processing
- Storing character data
- Pattern matching

Common pitfalls:

- Immutability means operations create new strings
- String concatenation in loops is inefficient
- Character-by-character processing can be slow

Example Problem: Valid Palindrome

Given a string, determine if it's a palindrome, considering only alphanumeric characters and ignoring case.

```
python

def is_palindrome(s):
    # Clean the string - O(n)
    cleaned = ''.join(c.lower() for c in s if c.isalnum())

    # Check if palindrome - O(n)
    return cleaned == cleaned[::-1]

# Alternative two-pointer approach - O(n) time, O(1) space
def is_palindrome_optimized(s):
    # Convert to lowercase and clean in one pass
    chars = [c.lower() for c in s if c.isalnum()]

    # Two-pointer technique
    left, right = 0, len(chars) - 1
    while left < right:
        if chars[left] != chars[right]:
            return False
        left += 1
        right -= 1

    return True
```

Key insight: Clean the input first, then use two pointers to check for palindrome property without creating a reversed copy.

Linked Lists

In Python, we implement linked lists using classes:

python

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# Creating a linked list
head = ListNode(1)
head.next = ListNode(2)
head.next.next = ListNode(3)

# Traversing a linked list
def print_list(head):
    current = head
    while current:
        print(current.val, end=" -> ")
        current = current.next
    print("None")

# Inserting a node at position 2
def insert_at_position(head, position, value):
    if position == 0:
        return ListNode(value, head)

    count = 0
    current = head
    while current and count < position - 1:
        current = current.next
        count += 1

    if current:
        current.next = ListNode(value, current.next)

    return head
```

When to use:

- When frequent insertions/deletions are needed
- When memory allocation needs to be flexible
- When implementing certain data structures (queues, LRU caches)

Common pitfalls:

- No random access (must traverse from head)
- Extra memory for pointers
- Easy to introduce cycles accidentally

Example Problem: Reverse Linked List

Reverse a singly linked list.

python

```
def reverse_list(head):
    prev = None
    current = head

    while current:
        next_temp = current.next # Store next node
        current.next = prev      # Reverse the pointer
        prev = current           # Move prev forward
        current = next_temp      # Move current forward

    return prev # New head is the previous last node

# Recursive approach
def reverse_list_recursive(head):
    # Base case: empty list or single node
    if not head or not head.next:
        return head

    # Recursive case: reverse rest of list
    new_head = reverse_list_recursive(head.next)

    # Current node's next node should point back to current node
    head.next.next = head

    # Current node should point to None (will be updated in next recursive call)
    head.next = None

    return new_head
```

Key insight: Track three pointers (prev, current, next) and carefully update references.

Stacks and Queues

Implemented using list or collections.deque:


```
python
```

```
# Stack using list
```

```
stack = []
```

```
stack.append(1) # Push - O(1)
```

```
stack.append(2)
```

```
top = stack[-1] # Peek - O(1)
```

```
stack.pop()    # Pop - O(1)
```

```
# Queue using collections.deque
```

```
from collections import deque
```

```
queue = deque()
```

```
queue.append(1) # Enqueue - O(1)
```

```
queue.append(2)
```

```
front = queue[0] # Peek front - O(1)
```

```
queue.popleft() # Dequeue - O(1)
```

When to use stacks:

- Function call tracking (recursive algorithms)
- Expression evaluation and syntax parsing
- Backtracking algorithms

When to use queues:

- Breadth-first search
- Task scheduling
- Resource pool management

Common pitfalls:

- Forgetting to check if empty before operations
- Using lists as queues (inefficient)
- Stack overflow in recursive implementations

Example Problem: Valid Parentheses

Given a string containing just the characters `(, ), {, }, [, and ]`, determine if the input string is valid.

python

```
def is_valid(s):
    stack = []
    mapping = {')': '(', '}': '{', ']': '['}

    for char in s:
        # If opening bracket, push to stack
        if char in "({[" :
            stack.append(char)
        # If closing bracket, check if matches top of stack
        elif char in ")}]":
            if not stack or stack.pop() != mapping[char]:
                return False

    # Stack should be empty if all brackets matched
    return len(stack) == 0
```

Key insight: Use a stack to track opening brackets and ensure they match with corresponding closing brackets.

Dictionaries (Hash Maps)

Python's built-in dictionaries implement hash tables:

python

```
# Creating and accessing
student = {"name": "John", "age": 21, "major": "CS"}
name = student["name"] # O(1) average case

# Check for key existence
has_name = "name" in student # O(1) average case

# Adding and removing items
student["gpa"] = 3.7 # O(1) average case
del student["age"] # O(1) average case

# Dictionary methods
keys = student.keys()
values = student.values()
items = student.items() # Returns (key, value) tuples

# Get with default
gpa = student.get("gpa", 0.0) # Returns 0.0 if key not found

# Dictionary comprehension
square_map = {x: x*x for x in range(5)} # {0:0, 1:1, 2:4, 3:9, 4:16}
```

When to use:

- Fast lookups by key
- Counting elements
- Caching/memoization
- Grouping data

Common pitfalls:

- Keys must be hashable (immutable)
- Order not guaranteed (before Python 3.7)
- Memory overhead
- Hash collisions can degrade performance

Example Problem: First Unique Character

Given a string, find the first non-repeating character and return its index.

```
python

def first_uniq_char(s):
    # Count frequency of each character
    char_count = {}
    for char in s:
        char_count[char] = char_count.get(char, 0) + 1

    # Find first character with count 1
    for i, char in enumerate(s):
        if char_count[char] == 1:
            return i

    return -1
```

Key insight: Use a hash map to count occurrences, then do a second pass to find the first unique character.

Sets

Python's sets implement hash tables for unique elements:

python

Creating sets

```
fruits = {"apple", "banana", "cherry"}
```

```
numbers = set([1, 2, 3, 3, 4]) # Duplicates removed: {1, 2, 3, 4}
```

Set operations

```
fruits.add("orange") # O(1) average case
```

```
fruits.remove("banana") # O(1) average case
```

```
is_present = "apple" in fruits # O(1) average case
```

Set methods

```
union = fruits.union({"grape", "kiwi"})
```

```
intersection = numbers.intersection({2, 3, 5})
```

```
difference = numbers.difference({2, 5})
```

Set comprehension

```
even_squares = {x*x for x in range(10) if x % 2 == 0}
```

When to use:

- Removing duplicates
- Membership testing
- Mathematical set operations
- Finding unique elements

Common pitfalls:

- Elements must be hashable
- No indexing/ordering (use OrderedDict for that)
- Cannot contain mutable objects as elements

Example Problem: Contains Duplicate

Given an array of integers, check if it contains any duplicate.

python

```
def contains_duplicate(nums):  
    # Using a set to track seen numbers  
    seen = set()  
    for num in nums:  
        if num in seen:  
            return True  
        seen.add(num)  
    return False  
  
    # One-liner using set comparison  
def contains_duplicate_oneshot(nums):  
    return len(set(nums)) < len(nums)
```

Key insight: A set gives $O(1)$ lookups and automatically handles duplicates.

Trees

Binary Trees, Binary Search Trees (BST), and more:

python

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

Creating a binary tree

```
root = TreeNode(1)
root.left = TreeNode(2)
root.right = TreeNode(3)
root.left.left = TreeNode(4)
root.left.right = TreeNode(5)
```

Tree traversals

```
def inorder(root):
    result = []
    if root:
        result.extend(inorder(root.left))
        result.append(root.val)
        result.extend(inorder(root.right))
    return result
```

```
def preorder(root):
    result = []
    if root:
        result.append(root.val)
        result.extend(preorder(root.left))
        result.extend(preorder(root.right))
    return result
```

```
def postorder(root):
    result = []
    if root:
        result.extend(postorder(root.left))
        result.extend(postorder(root.right))
        result.append(root.val)
    return result
```

BFS (level-order traversal)

```
def level_order(root):
    if not root:
        return []
```

```
    result = []
    queue = deque([root])
```

```
    while queue:
```

```
level = []
level_size = len(queue)

for _ in range(level_size):
    node = queue.popleft()
    level.append(node.val)

    if node.left:
        queue.append(node.left)
    if node.right:
        queue.append(node.right)

result.append(level)

return result
```

When to use:

- Hierarchical data relationships
- Searching in sorted data (BST)
- Expression parsing
- File system navigation

Common pitfalls:

- Forgetting base cases in recursive functions
- Unbalanced trees degrading to linked lists
- Not handling edge cases (empty trees)
- Improper tree modifications

Example Problem: Maximum Depth of Binary Tree

Find the maximum depth of a binary tree.

python

```
def max_depth(root):  
    # Base case: empty tree  
    if not root:  
        return 0  
  
    # Recursive case: 1 + max of left and right subtrees  
    left_depth = max_depth(root.left)  
    right_depth = max_depth(root.right)  
  
    return 1 + max(left_depth, right_depth)  
  
# Iterative solution using BFS  
def max_depth_iterative(root):  
    if not root:  
        return 0  
  
    queue = deque([(root, 1)]) # (node, depth)  
    max_depth = 0  
  
    while queue:  
        node, depth = queue.popleft()  
        max_depth = max(max_depth, depth)  
  
        if node.left:  
            queue.append((node.left, depth + 1))  
        if node.right:  
            queue.append((node.right, depth + 1))  
  
    return max_depth
```

Key insight: The depth of a tree is the maximum depth of its subtrees plus one.

Graphs

Represented using adjacency lists or matrices:

python

Adjacency list representation

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['A', 'D', 'E'],  
    'C': ['A', 'F'],  
    'D': ['B'],  
    'E': ['B', 'F'],  
    'F': ['C', 'E']  
}
```

DFS

```
def dfs(graph, start, visited=None):  
    if visited is None:  
        visited = set()  
  
    visited.add(start)  
    print(start, end=' ')  
  
    for neighbor in graph[start]:  
        if neighbor not in visited:  
            dfs(graph, neighbor, visited)  
  
    return visited
```

BFS

```
def bfs(graph, start):  
    visited = set([start])  
    queue = deque([start])  
    result = []  
  
    while queue:  
        vertex = queue.popleft()  
        result.append(vertex)  
  
        for neighbor in graph[vertex]:  
            if neighbor not in visited:  
                visited.add(neighbor)  
                queue.append(neighbor)  
  
    return result
```

When to use:

- Network connections
- Path finding algorithms
- Social networks
- Dependency relationships

Common pitfalls:

- Forgetting to track visited nodes (infinite loops)
- Memory overhead for large graphs
- Choosing wrong representation
- Incorrect edge handling in directed graphs

Example Problem: Number of Islands

Given a 2D grid map of '1's (land) and '0's (water), count the number of islands.

python

```
def num_islands(grid):
    if not grid or not grid[0]:
        return 0

    rows, cols = len(grid), len(grid[0])
    islands = 0

    def dfs(r, c):
        # Check bounds and if it's land
        if (r < 0 or r >= rows or
            c < 0 or c >= cols or
            grid[r][c] == '0'):
            return

        # Mark as visited by changing to '0'
        grid[r][c] = '0'

        # Check all 4 directions
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                islands += 1
                dfs(r, c) # Mark all connected land

    return islands
```

Key insight: Use DFS to find and mark connected components (islands).

Heaps (Priority Queues)

Python provides heapq for min-heap operations:

python

```
import heapq
```

Creating a heap

```
nums = [3, 1, 4, 1, 5, 9, 2, 6]
```

```
heapq.heapify(nums) #  $O(n)$  - convert to min-heap in-place
```

Operations

```
smallest = heapq.heappop(nums) #  $O(\log n)$  - remove and return smallest
```

```
heapq.heappush(nums, 7) #  $O(\log n)$  - add value to heap
```

Peek at smallest without removing

```
if nums:
```

```
    smallest = nums[0]
```

For max-heap, negate values

```
max_heap = [-x for x in [3, 1, 4, 1, 5, 9]]
```

```
heapq.heapify(max_heap)
```

```
largest = -heapq.heappop(max_heap) # Remove and return largest
```

Priority queue with (priority, item) tuples

```
pq = [(2, 'task2'), (5, 'task5'), (1, 'task1')]
```

```
heapq.heapify(pq)
```

```
highest_priority = heapq.heappop(pq) # (1, 'task1')
```

When to use:

- Implementing priority queues
- Finding k largest/smallest elements
- Scheduling based on priority
- Graph algorithms (Dijkstra's, Prim's)

Common pitfalls:

- Forgetting that heapq implements min-heap by default
- Modifying values without reheapifying
- Inefficient access to specific elements
- Misunderstanding how tuples are compared

Example Problem: Kth Largest Element

Find the kth largest element in an unsorted array.

python

```
def find_kth_largest(nums, k):
    # Using heap -  $O(n + k \log n)$ 
    import heapq

    # Convert to max-heap by negating values
    max_heap = [-num for num in nums]
    heapq.heapify(max_heap) #  $O(n)$ 

    # Pop k times to get kth largest
    for _ in range(k - 1):
        heapq.heappop(max_heap) #  $O(\log n)$ 

    return -heapq.heappop(max_heap)

# Alternative using min-heap
def find_kth_largest_min_heap(nums, k):
    import heapq

    # Keep a min-heap of k largest elements
    min_heap = nums[:k]
    heapq.heapify(min_heap) #  $O(k)$ 

    # Process remaining elements
    for num in nums[k:]:
        if num > min_heap[0]:
            heapq.heappushpop(min_heap, num) #  $O(\log k)$ 

    return min_heap[0]
```

Key insight: Use a heap to efficiently track the k largest elements.

Essential Algorithms

Searching

Linear Search

python

```
def linear_search(arr, target):
    for i, num in enumerate(arr):
        if num == target:
            return i
    return -1
```

Complexity: $O(n)$ time, $O(1)$ space

Binary Search

python

```
def binary_search(sorted_arr, target):  
    left, right = 0, len(sorted_arr) - 1  
  
    while left <= right:  
        mid = left + (right - left) // 2  
  
        if sorted_arr[mid] == target:  
            return mid  
        elif sorted_arr[mid] < target:  
            left = mid + 1  
        else:  
            right = mid - 1  
  
    return -1
```

Complexity: $O(\log n)$ time, $O(1)$ space

Binary Search Variations

python

Find first occurrence of target

```
def binary_search_first(sorted_arr, target):  
    left, right = 0, len(sorted_arr) - 1  
    result = -1  
  
    while left <= right:  
        mid = left + (right - left) // 2  
  
        if sorted_arr[mid] == target:  
            result = mid # Record this occurrence  
            right = mid - 1 # But keep searching to the left  
        elif sorted_arr[mid] < target:  
            left = mid + 1  
        else:  
            right = mid - 1  
  
    return result
```

Find last occurrence of target

```
def binary_search_last(sorted_arr, target):  
    left, right = 0, len(sorted_arr) - 1  
    result = -1  
  
    while left <= right:  
        mid = left + (right - left) // 2  
  
        if sorted_arr[mid] == target:  
            result = mid # Record this occurrence  
            left = mid + 1 # But keep searching to the right  
        elif sorted_arr[mid] < target:  
            left = mid + 1  
        else:  
            right = mid - 1  
  
    return result
```

Sorting

Quick Sort

python

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort(left) + middle + quick_sort(right)
```

Complexity: $O(n \log n)$ average, $O(n^2)$ worst-case time, $O(n)$ space

Merge Sort

python

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Complexity: $O(n \log n)$ time, $O(n)$ space

Custom Sorting

python

Sort by multiple criteria

```
students = [  
    ("Alice", 90, 22),  
    ("Bob", 85, 20),  
    ("Charlie", 90, 21)  
]
```

Sort by grade (descending), then by age (ascending)

```
students.sort(key=lambda x: (-x[1], x[2]))
```

Sort with custom comparison using functools.cmp_to_key

```
from functools import cmp_to_key
```

```
def custom_compare(a, b):
```

First compare by grade (descending)

```
if a[1] != b[1]:
```

```
    return b[1] - a[1]
```

If grades are equal, compare by age (ascending)

```
    return a[2] - b[2]
```

```
students.sort(key=cmp_to_key(custom_compare))
```

Graph Algorithms

Depth-First Search (DFS)

python

```
def dfs(graph, node, visited=None):
```

```
    if visited is None:
```

```
        visited = set()
```

```
    visited.add(node)
```

```
    print(node, end=' ')
```

```
    for neighbor in graph[node]:
```

```
        if neighbor not in visited:
```

```
            dfs(graph, neighbor, visited)
```

```
    return visited
```

Complexity: $O(V + E)$ time, $O(V)$ space, where V is vertices and E is edges

Breadth-First Search (BFS)

python

```
from collections import deque

def bfs(graph, start):
    visited = set([start])
    queue = deque([start])

    while queue:
        node = queue.popleft()
        print(node, end=' ')

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return visited
```

Complexity: $O(V + E)$ time, $O(V)$ space

Dijkstra's Algorithm (Shortest Path)

python

```
import heapq

def dijkstra(graph, start):
    # Initialize distances with infinity for all nodes except start
    distances = {node: float('infinity') for node in graph}
    distances[start] = 0
    priority_queue = [(0, start)]

    while priority_queue:
        current_distance, current_node = heapq.heappop(priority_queue)

        # If we've found a longer path, ignore it
        if current_distance > distances[current_node]:
            continue

        # Check all neighbors
        for neighbor, weight in graph[current_node].items():
            distance = current_distance + weight

            # If we've found a shorter path, update it
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                heapq.heappush(priority_queue, (distance, neighbor))

    return distances
```

Complexity: $O((V + E) \log V)$ time, $O(V)$ space

Dynamic Programming

Fibonacci with Memoization

python

```
def fibonacci(n, memo={}):
    if n in memo:
        return memo[n]

    if n <= 1:
        return n

    memo[n] = fibonacci(n-1, memo) + fibonacci(n-2, memo)
    return memo[n]
```

Complexity: $O(n)$ time, $O(n)$ space

Bottom-Up Dynamic Programming

python

```
def fibonacci_bottom_up(n):  
    if n <= 1:  
        return n  
  
    dp = [0] * (n + 1)  
    dp[1] = 1  
  
    for i in range(2, n + 1):  
        dp[i] =
```